

A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is a light greenish-blue. They are positioned diagonally, with the blue one partially covering the green one.

09 - Interprocess Communication



Why we need semaphores

- What do we do if we have two programs that need to communicate with each other?
 - For now, assume they're on the same machine
- We can technically use files, but there are pitfalls:
 - Permissions
 - Make sure unwanted processes cannot access them
 - Accidental overwrites
 - Efficiency
- Shared memory is more efficient, but we need to deal with race conditions



Semaphores

- A kernel-maintained integer that can be operated on by multiple processes via a queue-like system.
- Includes the rest of the data that allows these operations
- It is technically a shared resource.



Semaphores

- Recall the *busy waiting* version of a Semaphore:

```
void wait (Semaphore s) {    //atomic?
    while (s <= 0) {}    // NOP
    s -= 1;
}

void signal (Semaphore s) { //atomic!
    s += 1;
}
```



Semaphores

- UNIX Systems have two APIs for semaphores
 - System V Semaphores which is heavyweight, creating a set or array of semaphores
 - POSIX Semaphores which is lightweight, creating only one semaphore.
- We will focus on the first one (System V Semaphores).

A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is light green. They are positioned diagonally, with the blue one partially covering the green one.

System V Semaphores



System V Semaphores

- Download the file `sem_sample.cpp`



System V Semaphores

STEP 1: Initialization

```
int semget (key_t key, int nsems, int semflg)
```

- System call that creates a new semaphore set, or obtains the identifier of an existing semaphore set.
- On success, `semget()` returns the ID for the new or existing semaphore set.
 - Otherwise returns -1 and sets the value of `errno` to the corresponding error.



System V Semaphores

STEP 1: Initialization

```
int semget (key_t key, int nsems, int semflg)
```

- **key_t key** is a key associated with a semaphore set. Can be a “magic” number, the value **IPC_PRIVATE**, or a key returned by **ftok()**.
- **int nsems** specifies the number of semaphores in the set to be created. If it already exists, **nsems** should be less than or equal to the size of the set. Passing 0 guarantees success.
- **int semflg** is a bit mask containing flags and permissions



System V Semaphores

- Different programs may use the same key so the `semget()` calls will return the same semaphore set.
 - This is an ever-present risk, especially if everyone is using a central server for their programs.
- System V Semaphores have various system calls, including:
 - Adding or subtracting a number to the value of the semaphore, and
 - Waiting for a semaphore to be equal to 0.
- These are done through the `semop()` function.



System V Semaphores

STEP 2: Accessing

```
int semop (int semid, struct sembuf* sops,  
unsigned int nsops)
```

- System call that performs one or more operations on the semaphore set, specified by **semid**.
- **sops** is a pointer to an array of operations to be performed
- **nsops** is the size of the sops array and must be at least 1.



System V Semaphores

- The array `sops` contains structs that are defined as:

```
struct sembuf {  
    unsigned short sem_num; // Semaphore number  
    short sem_op;           // Operation  
    short sem_flg;         // Operation flags  
    // IPC_NOWAIT, SEM_UNDO  
}
```



System V Semaphores

STEP 2: Accessing

- If `sem_op`:
 - `>0`, the value of `sem_op` is added to the current value of the semaphore.
 - `=0`, the value of the semaphore is checked if zero. If yes, the operation completes immediately. Else, `semop( )` blocks.
 - `<0`, decrease the value of the semaphore by `sem_op`. `semop( )` may block if the resulting value is negative.



System V Semaphores

STEP 2: Accessing

- **SEM_UNDO**: Allocates space for an undo operation if the process terminates abnormally. Prevents semaphores from accidentally being locked forever, or until a system reboot.
- **IPC_NOWAIT**: Force error if operation must wait.



System V Semaphores

STEP 2: Accessing

- Each call to `semop( )` is atomic, so all the operations in sops are performed sequentially in array order. And if one blocks, all operations block.
- When `semop( )` blocks, it stay blocked until:
 - Another process modifies the value of the semaphore such that a requested operation can now be performed, or
 - A signal interrupts semop() (EINTR error), or
 - Another process deletes the semaphore set referred to by `semid` (EIDRM error)



System V Semaphores

STEP 3: Reading and Writing

- At this point the process/thread can access the shared resource that the semaphore was holding and do whatever it needs to do.
 - Of course because a shared resource is involved the process has to let go of the semaphore afterwards.



System V Semaphores

STEP 4: Releasing

- To release, just call `semop()` again, but instead of incrementing, we decrement it.
 - Essentially set the `sem_op` value of the `sembuf` struct to a negative number (in this case, -1).
- After that, the other processes that were waiting can now gain control of it.
- Ponder: what resource should the semaphore be guarding?

A decorative graphic on the left side of the slide. It consists of a blue parallelogram and a light green parallelogram, both tilted at an angle. The blue shape is in the foreground, and the green shape is partially behind it. They are set against a dark blue background with faint, lighter blue diagonal stripes.

Shared Memory



Shared Memory

- (Download the file `shm_sample.cpp`)
- Shared memory can be accessed like a string or array of chars
- Also has permission bits just like files and semaphores
 - But this doesn't make an actual file that you can search for in the file system.
- This next bit will look almost identical to semaphores...



Shared Memory

STEP 1: Initialization

```
int shmget (key_t key, size_t size, int shmflg)
```

- System call that creates a new shared memory set, or obtains the identifier of an existing shared memory set.
- On success, `shmget()` returns the ID for the new or existing shared memory segment.
 - Otherwise returns -1 and sets the value of `errno` to the corresponding error.



Shared Memory

STEP 1: Initialization

```
int shmget (key_t key, size_t size, int shmflg)
```

- **key_t key** is a key associated with shared memory segment. Can be a “magic” number, the value **IPC_PRIVATE**, or a key returned by **ftok()**.
- **size_t size** specifies the size of the shared memory in bytes. A size that is a power of 2 is usually preferred.
- **int shmflg** is a bit mask containing flags and permissions.



Shared Memory

- Different programs may use the same key so the `shmget()` calls will return the same shared memory set.
 - This is an ever-present risk, especially if everyone is using a central server for their programs.



Shared Memory

STEP 2: Accessing

```
void* shmat (int shmid, const void* smaddr,  
int shmflag)
```

- System call for attaching the shared memory segment associated with `shmid` to the address space of the calling process.
- On success, it returns a pointer to the starting address of the shared memory



Shared Memory

STEP 2: Accessing

```
void* shmat (int shmid, const void* smaddr,  
int shmflag)
```

- On success, it returns a pointer to the starting address of the shared memory
 - Otherwise, the address is not attached, it returns -1 and sets the value of `errno` to the corresponding error.



Shared Memory

STEP 2: Accessing

```
void* shmat (int shmid, const void* smaddr,  
int shmflag)
```

- `int shmid` is the id of the shared memory segment that you want to attach (e.g. the return value of the prev `shmget()`)



Shared Memory

STEP 2: Accessing

```
void* shmat (int shmid, const void* smaddr,  
int shmflag)
```

- **const void* smaddr** is the starting address of the shared memory segment
 - If it's NULL, then it's attached to the first available address according to the system
 - Otherwise, depends on the flags specified.



Shared Memory

STEP 2: Accessing

```
void* shmat (int shmid, const void* smaddr,  
int shmflag)
```

- **int shmflag** is a bit mask containing flags that dictates the behavior of the function. We usually set this to 0 unless we have special requirements for the shared memory segment.



Shared Memory

STEP 3: Reading and Writing

- Given a pointer to the address of the shared memory, you may now start writing and reading from it through:
 - `strcpy(char* dest, char* src)`
 - `memcpy(void* dest, const void* src, size_t count)`
 - And other functions..



Shared Memory

And that's basically it,

- Semaphores + Shared Memory = Local IPC
- Program should look like:
 - Initialize shared resources (semaphores and shared mem)
 - Gain control of the semaphore
 - Access shared memory
 - Release the semaphore